TECHNICAL OVERVIEW

Jarvin: how it works

A voice assistant that runs on my own laptop and does real work on Windows.



Navaz Chilakala · November 2026

WHAT IT IS

Three things set it apart

01

It acts

Opens apps, presses buttons, types, reads the screen, makes documents, sets reminders and edits code.

02

It runs locally

Chat, vision and speech models run on the laptop’s own graphics card through Ollama.

03

It is supervised

Every action passes a permission check in code. Risky ones wait for approval.

Example: “Open Calculator and work out 12 times 12” opens the real Calculator, presses the real buttons and reads back 144.

A language model only produces text. Everything else is ordinary code around it.

LANGUAGE MODELS

Four facts that shape the design

It reads tokens

A token is a piece of a word. Everything the model reads costs time.

It has no memory

Each call is independent, so the whole conversation is sent again every time.

It can only read so much

The context window caps one call. Jarvin’s default is 16,384 tokens.

It only writes text

It can ask for an action. A program has to carry it out.

RUNNING LOCALLY

Graphics memory sets the speed

2.2 s

One model

A single model that can both chat and see.

3.3 s

Two models that fit

Chat and vision both stay loaded.

15–18 s

Two that must swap

The current setup: qwen3:8b for chat, qwen3-vl for vision.

Time per screen step on an 8 GB laptop GPU. The better chat model cannot see, so screen tasks pay for a second model.

TOOL CALLING

From chatbot to agent

1
Send the tool list
Each tool has a name and a plain description.

2
Model asks for a tool
launch_app with app “Calculator”.

3
Program runs it
Real code does the work and gets a result.

4
Result goes back
The model reads it and picks the next step.

Steps 2 to 4 repeat until the model answers in plain words. That repetition is the agent loop.

PART B · THE REAL SYSTEM

Four layers, each with one job

Windows show things	Main window, floating pet and voice overlay. Web pages that only display events and send named messages.
Main process decides things	Agent loop, about 100 tools, permissions and sandbox, voice service, and the stores for chats and recovery.
Helpers do risky work	PowerShell desktop worker, speech recogniser, voice synthesiser, document workers and a crash watcher.
Outside Jarvin	Ollama with the chat and vision models, Windows apps, the web, and optional MCP servers and cloud models.

ONE REQUEST, END TO END

“Open Calculator and work out 12 times 12”

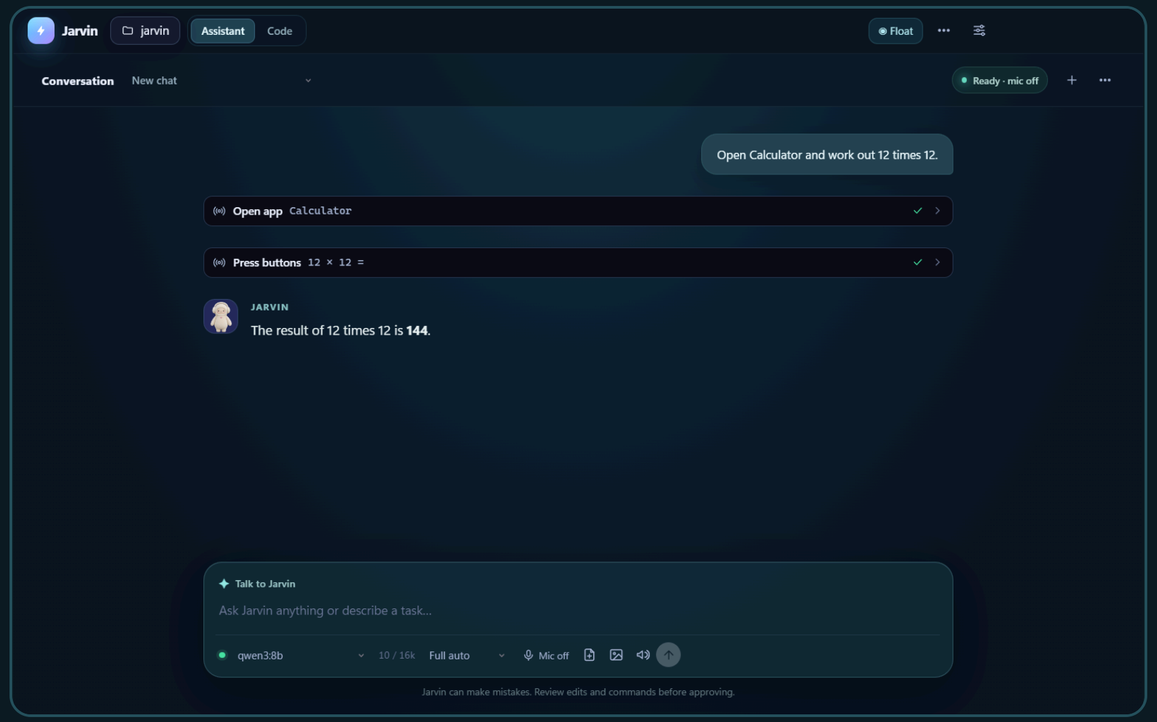
1. The window sends the request to the main process.

2. The run is journalled, checkpointed and traced.

3. The model asks for launch_app. Permission check, then the app opens.

4. The model asks for click_controls. The desktop worker presses 1, 2, ×, 1, 2, =.

5. The worker reads the display. The model answers: 144.



THE AGENT LOOP

A simple loop, wrapped in guards

Guard	What it prevents
A step limit pauses the run	A task that never ends
Three identical calls in a row pause it	A model stuck on a failing action
A promise with no action gets a nudge	Models that announce a step and then stop
Reads run together, writes run in order	One action overtaking another
Output is capped and secrets are masked	Flooding the context or the log
Every result is labelled untrusted	Instructions hidden in files or web pages

Each guard exists because of a real failure seen during testing.

TOOLS

Send only the tools a request needs

~100

Tools built in

Apps and windows, typing, seeing, documents, web, reminders, memory, files and coding.

28

For a plain desktop request

A request with desktop words and no coding words gets the short core list.

ADDED ON DEMAND

Extra groups by keyword

“Search” adds web tools. “Excel” adds document tools. “Remind” adds reminder tools.

Every tool has a risk class: read, write or execute. That class drives the permission check.

SPEED

Keep the start of every call identical

0.3 s

Follow-up request

Time before the answer starts.

2.3 s

First request after start-up.

HOW

The clock moved. Date and time go at the end of the user's message, not at the top of the system prompt.

One tool list. The same code builds it for real calls and for warm-up, so they match byte for byte.

Priming at start-up. The chat model reads the real prompt once before the first request.

Chat model warmed last, so it is the one left in memory.

CONTROLLING THE DESKTOP

Accessibility first, eyes second

FAST PATH

Read the controls by name

Windows UI Automation exposes a window as named controls. Jarvin presses “One”, “Two”, “Multiply by” directly and reads the display back. No screenshot.

GENERAL PATH

Look, then click

Capture the window, ask the local vision model where the target is, send a real mouse click, then look again to confirm.

One action at a time

Requests to the desktop worker queue.

Same window, checked

Identity is re-verified before each action.

Never retry blindly

A click that may have landed is not repeated.

VOICE IN

Five gates before speech becomes action

1

Mic switch

Off at start. Nothing is recorded until it is on.

→

2

Segmenter

Cuts the stream into one utterance.

→

3

Recogniser

Qwen3-ASR turns it into words, on the laptop.

→

4

Wake word

Must start with “Hey Jarvin”.

→

5

Task rule

While working, only “stop” gets through.

“Stop” needs no name. Jarvin also drops anything that matches its own recent speech.

VOICE IN · EVIDENCE

The recogniser was chosen by test

Recogniser	Wrong words of 160	“Hey Jarvin” heard
Qwen3-ASR with a name hint (chosen)	1	10 of 10
Qwen3-ASR without the hint	3	9 of 10
distil-small.en	13	4 of 10
whisper-base.en	13	1 of 10
moonshine-base	20	1 of 10

20 fixed sentences recorded in my own voice, then every installed recogniser run over the same recordings.

SAFETY

Permission is decided in code

Checked in this order	Outcome
1. A command on the dangerous list	Blocked
2. A deny rule matches	Blocked
3. Sensitive path, secret-like text, delete or screenshot	Always asks
4. The tool only reads	Allowed
5. A saved “always allow” rule matches	Allowed
6. Anything else	The approval mode decides

Modes: Ask first · Auto desktop · Full auto · Plan only. A project can add block rules but can never loosen them.

RECOVERY

Mistakes and crashes are expected

Checkpoints

Originals are saved before a file first changes in a run. Any run can be rewound.

Run journal

Each run is recorded. An interrupted task is offered again after a restart, never replayed blindly.

Crash handling

A crashed window is reloaded. A small watcher process restarts the whole app.

Traces

Every event of a run is logged with secrets masked, so a wrong action can be replayed and understood.

KNOWN LIMITS

What is not solved yet

Unsigned files

Windows Smart App Control blocks the installer here, and on some launches the speech engine. The lasting fix is a code-signing certificate.

Slow screen tasks

15–18 s per screen step with the current two-model setup on 8 GB of graphics memory.

Loudness-based listening

A loud non-speech sound can start an utterance. A trained speech detector is the planned next step.

Small-model mistakes

An 8B local model sometimes reports what it expected instead of what it saw, or needs a retry.

THE SHAPE TO REMEMBER

The window shows.
The main process decides.
Helpers do the risky work.
The model only suggests.

Source and full technical guide: github.com/Navaz-Chilakala/jarvin